Status	Finished
Started	Friday, 25 September 2026, 10:32 AM
Completed	Friday, 25 September 2026, 10:51 AM
Duration	19 mins 4 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

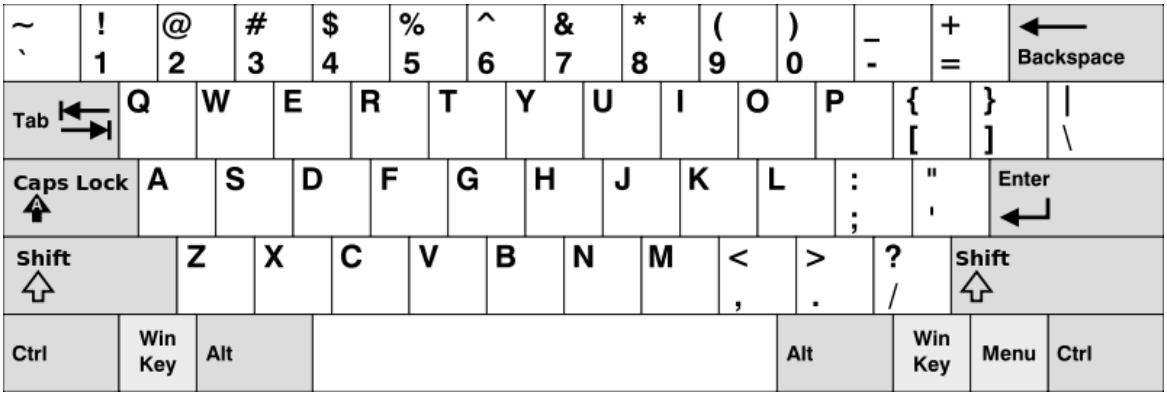
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    row1 = set("qwertyuiop")
    row2 = set("asdfghjkl")
    row3 = set("zxcvbnm")

    idx = 0
```

```
    while idx < len(input_data):
        try:
            # Read the number of words for the current test case
            n = int(input_data[idx])
            idx += 1
        except ValueError:
            break

        # Extract the words for this test case
        words = input_data[idx:idx + n]
        idx += n

        result = []
        for word in words:
            w_set = set(word.lower())
            if w_set.issubset(row1) or w_set.issubset(row2) or w_set.issubset(row3):
                result.append(word)

        # Print results or "No words" if empty
        if result:
            for w in result:
                print(w)
        else:
            print("No words")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    if not lines:
        return

    # Process lines in pairs: text and brokenLetters
    i = 0
    while i < len(lines):
        text = lines[i].strip()
        if not text:
            i += 1
            continue

        broken = lines[i+1].strip() if i + 1 < len(lines) else ""
        i += 2

        # Convert broken letters to a set of lowercase characters for case-insensitive checking
        broken_set = set(broken.lower())
        words = text.split()

        count = 0
        for word in words:
            # Check if any character in the word (converted to lowercase) is broken
            if not any(char.lower() in broken_set for char in word):
                count += 1

        print(count)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓

	Input	Expected	Got	
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) < 2:
        return

    # Parse the tuple elements from the first line (comma-separated)
    t = tuple(map(int, lines[0].split(',')))
    # Parse K from the second line
    k = int(lines[1])

    distinct_pairs = set()
    n = len(t)

    # Check all unique index pairs (i, j) where i < j
    for i in range(n):
        for j in range(i + 1, n):
            if t[i] + t[j] == k:
                # Sort the pair values to treat (a, b) and (b, a) as the same distinct pair
                pair = tuple(sorted((t[i], t[j])))
                distinct_pairs.add(pair)

    # Print the total count of distinct pairs
    print(len(distinct_pairs))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from standard input (handles both space and newline separations)
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First token is the array length n
    n = int(input_data[0])

    # Extract the next n elements as the array
    arr = input_data[1:1 + n]

    # Extract unique elements while preserving order of appearance
    distinct_elements = []
    seen = set()
    for x in arr:
        if x not in seen:
            seen.add(x)
            distinct_elements.append(x)

    # Print the distinct elements separated by a single space
    print(" ".join(distinct_elements))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the entire input string from standard input
    s = sys.stdin.read().strip()
    if not s:
        print("No")
        return

    # Create a set of characters in the string
    char_set = set(s)

    # Define the valid set of characters for a binary string
    binary_set = {'0', '1'}

    # Check if all characters in the string are strictly '0' or '1'
    if char_set.issubset(binary_set):
        print("Yes")
    else:
        print("No")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) < 3:
        return

    # Parse the two arrays from lines 1 and 2
    arr1 = list(map(int, lines[1].split()))
    arr2 = list(map(int, lines[2].split()))

    set1 = set(arr1)
    set2 = set(arr2)

    # Find elements that are in either set1 or set2, but not in both (symmetric difference)
    non_rep_set = set1 ^ set2

    if not non_rep_set:
        print("NO SUCH ELEMENTS")
        return

    # Collect non-repeating elements while preserving their original order of appearance
    result = []
    seen = set()
```

```
    for x in arr1 + arr2:
        if x in non_rep_set and x not in seen:
            seen.add(x)
            result.append(x)

    # Print the space-separated non-repeating elements and their count
    print(" ".join(map(str, result)))
    print(len(result))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all space-separated tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    nums = list(map(int, input_data))
    seen = set()

    # Iterate through the numbers and check for duplicates using a set
    for num in nums:
        if num in seen:
            print(num)
            return
        seen.add(num)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
from collections import deque

def count_operations(nums):
    d = deque(nums)
    ops = 0

    while d:
        ops += 1
        # If the first element is the minimum in the current collection, remove it
        if d[0] == min(d):
            d.popleft()
        else:
            # Otherwise, move the first element to the end
            d.append(d.popleft())

    return ops
```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓

	Test	Expected	Got	
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.strip()]
    if len(lines) < 3:
        return

    # Parse each line into a set of words
    set1 = set(lines[0].split())
    set2 = set(lines[1].split())
    set3 = set(lines[2].split())

    # Check if set3 is a subset of both set1 and set2
    if set3.issubset(set1) and set3.issubset(set2):
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("No")

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the **`10`-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the DNA sequence string from standard input
    s = sys.stdin.read().strip()
    if not s or len(s) < 10:
        return

    seen = set()
    repeated = set()

    # Use a sliding window of length 10
    for i in range(len(s) - 9):
        substring = s[i:i + 10]
        if substring in seen:
            repeated.add(substring)
        else:
        # Optimization: use separate tracking to avoid duplicate prints
            seen.add(substring)

    # Print each unique repeated 10-letter sequence on a new line
    for seq in repeated:
        print(seq)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA	AAAAACCCCC CCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.